FIELD NOTES

BigCommerce Field Guide

BigCommerce order, webhook and catalog fixes.

94 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 94 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/bigcommerce-fixes
Guides: allanninal.dev/bigcommerce
All 14 repos: github.com/allanninal

The fixes

1 **422 fulfillment address incomplete despite address looking complete**
The consignment call succeeded. The checkout looks fully addressed. Then placing the order comes back 422, a fulfillment address for this order is incomplete. BigCommerce validates fulfillment addresses far more strictly at order-creation time than the create-consignment call ever hinted at, so one missing leaf key slips through unnoticed until it blocks the order. Here is why that gap opens up and a script that names the exact missing subfield per order or checkout.
`find_incomplete_fulfillment_addresses.py`
https://www.allanninal.dev/bigcommerce/422-fulfillment-address-incomplete/
https://github.com/allanninal/bigcommerce-fixes/tree/main/422-fulfillment-address-incomplete

---

2 **429 responses sometimes omit rate limit headers under high load**
A normal BigCommerce 429 comes with four headers that tell you exactly how long to wait. But when the platform itself is under high load, not just your own store's quota, the edge layer can return a bare 429 with none of those headers attached. Scripts that expect X-Rate-Limit-Time-Reset-Ms and choke on its absence end up retrying immediately in a hot loop, which makes the overload worse. Here is why the headers go missing and a small backoff helper that never depends on them.
`rate_limit_backoff.py`
https://www.allanninal.dev/bigcommerce/429-missing-rate-limit-headers/
https://github.com/allanninal/bigcommerce-fixes/tree/main/429-missing-rate-limit-headers

---

3 **Abandoned cart records pile up in BigCommerce**

Open the cart list in a store that has been live for a year or two and the count is startling. Thousands of records, most of them empty or long dead, and no button anywhere that clears them out. BigCommerce carts are created liberally by guest checkouts, recovery flows, headless sessions, and apps, and none of them ever expire or self-delete on the server. Here is why the pile keeps growing and a small script that classifies the mess and cleans up only what is truly safe to remove.

`clean_stale_carts.py`

https://www.allanninal.dev/bigcommerce/abandoned-cart-records-pile-up/

https://github.com/allanninal/bigcommerce-fixes/tree/main/abandoned-cart-records-pile-up

---

4 **Applying an API discount clears existing order discounts**

A script posts a new discount to add a promo code, and the customer's original coupon just vanishes from the cart. Nothing in the response says anything went wrong. BigCommerce's checkout-discounts endpoint replaces the whole discount set on every call instead of adding to it, so a well-intentioned integration can silently wipe every coupon, automatic promotion, and prior manual discount already sitting on the checkout. Here is why that happens, how to catch it before the order is placed, and a script that reports every checkout it happened to.

`detect_cleared_discounts.py`

https://www.allanninal.dev/bigcommerce/api-discount-clears-existing-discounts/

https://github.com/allanninal/bigcommerce-fixes/tree/main/api-discount-clears-existing-discounts

---

5 **Order created via API bypasses customer group and price list pricing**

A wholesale customer has a price list assigned to their customer group and gets the right discount at storefront checkout every time. Then an integration creates the same customer's order through the API, and the invoice comes back at full retail. POST /v2/orders is a back-office order-entry endpoint, not the checkout pricing engine, and when the caller hands it a line price it trusts that number completely. Here is why that gap opens up and a script that finds every order billed at the wrong price.

`diagnose_order_pricing.py`

https://www.allanninal.dev/bigcommerce/api-order-bypasses-group-pricing/

https://github.com/allanninal/bigcommerce-fixes/tree/main/api-order-bypasses-group-pricing

---

6 **Available drifts from real on-hand**

A cycle count comes back from the warehouse and a SKU that BigCommerce shows as 12 available is really 7 on the shelf, or the other way around. Nobody changed anything on purpose. The storefront number is only ever as good as the last write to it, and orders, cancellations, refunds, and bulk imports all write to it independently. Here is why that number quietly drifts and a small script that pulls a real count, compares it to what BigCommerce has, and reconciles only the SKUs that actually need it.

`reconcile_inventory.py`

https://www.allanninal.dev/bigcommerce/available-drifts-from-real-on-hand/

https://github.com/allanninal/bigcommerce-fixes/tree/main/available-drifts-from-real-on-hand

---

7 **Cart contents lost across devices or sessions in the B2B buyer portal**

A wholesale buyer builds a big cart on their desktop at the office, then opens the B2B Buyer Portal on their phone in the warehouse and finds it empty. Nothing was deleted. A brand new anonymous cart was simply created in its place, because BigCommerce has no way to look up a customer's existing cart from a new device. Here is why that gap exists and a script that finds the orphaned duplicate carts piling up behind it, so you can see the damage before you touch anything.

`reconcile_b2b_carts.py`

https://www.allanninal.dev/bigcommerce/b2b-cart-not-persisted-across-sessions/

https://github.com/allanninal/bigcommerce-fixes/tree/main/b2b-cart-not-persisted-across-sessions

---

8 **Backfill order metadata for matching**

Before the ERP or marketplace integration was wired up, orders came in with no external_id, no external_merchant_id, and no external_source. Now a reconciliation job needs to join those old BigCommerce orders to ERP records and has nothing solid to join on. You cannot just PUT the missing fields back in, BigCommerce will not accept it. Here is why, and a small script that writes a safe reconciliation key somewhere BigCommerce will actually let you keep it.

`backfill_order_metadata.py`

https://www.allanninal.dev/bigcommerce/backfill-order-metadata-for-matching/

https://github.com/allanninal/bigcommerce-fixes/tree/main/backfill-order-metadata-for-matching

---

9 **BigCommerce brand update immediately before product create returns empty reply**

You PUT an update to a brand, then POST a new product against it right away, and the client comes back with nothing. No status code, no JSON, just "Empty reply from server." BigCommerce's rate limit and concurrency cap can close the connection mid-response when the two calls land back to back, and the brand update or the product create may have actually gone through on BigCommerce's side even though your script never saw a usable answer. Here is why that gap opens up and a small script that reconciles the pair against real catalog state instead of guessing.

`reconcile_brand_product_race.py`

https://www.allanninal.dev/bigcommerce/brand-update-product-create-race/

https://github.com/allanninal/bigcommerce-fixes/tree/main/brand-update-product-create-race

---

10 **Broken product images in BigCommerce**

A product page loads fine, the title and price are right, but the picture is a broken icon. BigCommerce still returns what looks like a valid image object from its API. The row in the catalog is there. The file it points to is not. Here is why an image record can outlive its own file and a small script that finds every one of these and fixes it without guessing.

`find_broken_images.py`

https://www.allanninal.dev/bigcommerce/broken-product-images/

https://github.com/allanninal/bigcommerce-fixes/tree/main/broken-product-images

---

11 **BigCommerce bulk image API only persists the first image per request**

The import script uploads five images for a product and logs five 200s, but the product page in the admin only ever shows one. There is no bug in your loop. BigCommerce's v3 catalog images endpoint was never built to take a batch, it takes exactly one image per call, and a script written by analogy to the batch products or variants endpoints will quietly lose every image after the first. Here is why that gap opens up and a small reconciler that finds the missing images per product and requeues only those.

`requeue_missing_images.py`

https://www.allanninal.dev/bigcommerce/bulk-image-api-single-image-per-request/

https://github.com/allanninal/bigcommerce-fixes/tree/main/bulk-image-api-single-image-per-request

---

12 **Cart stays locked to its original currency after a currency switch**

A shopper adds a shirt while browsing in USD, then switches the storefront currency selector to EUR. The page redraws every price in euros, but the cart underneath never got the memo. BigCommerce fixes a cart's transactional currency at the moment it is created, and there is no API call that can change it afterward. The switch only updates a display preference, so the cart, and checkout behind it, keeps charging in the original currency. Here is why that gap exists and a script that finds the carts it left behind.

`find_currency_mismatched_carts.py`

https://www.allanninal.dev/bigcommerce/cart-locked-to-original-currency/

https://github.com/allanninal/bigcommerce-fixes/tree/main/cart-locked-to-original-currency

---

13 **BigCommerce category image_file field rejected as invalid on update**

A sync script PUTs a category update with an image_file field and BigCommerce answers with a flat 400, the field 'image_file' is invalid. The category's image never changes, and depending on the payload the whole call can fail. The cause is simple once you see it: the V3 Catalog Categories JSON endpoint only ever accepts image_url. image_file only exists on a separate multi-part endpoint. Here is why the two get confused and a small script that picks the right one automatically.

`repair_category_images.py`

https://www.allanninal.dev/bigcommerce/category-image-file-rejected/

https://github.com/allanninal/bigcommerce-fixes/tree/main/category-image-file-rejected

---

14 **Concurrent inventory and catalog or order bulk jobs corrupt stock totals**

Two bulk jobs, one SKU, one location, the same window of time. An inventory adjustment job and a catalog or order bulk job both touch the same underlying stock counter, and BigCommerce's own documentation warns this can produce unpredictable, incorrect results. The two writes race, one silently clobbers or double-applies the other, and total_inventory_onhand quietly drifts from what your store actually has. Here is why the race happens and a script that detects the drift against your own adjustment ledger and repairs it safely.

`reconcile_concurrent_inventory_drift.py`

https://www.allanninal.dev/bigcommerce/concurrent-inventory-catalog-jobs-corrupt-stock/

https://github.com/allanninal/bigcommerce-fixes/tree/main/concurrent-inventory-catalog-jobs-corrupt-stock

---

15 **Concurrent price list bulk upserts fail the whole batch with 429**

A nightly sync fans out one bulk upsert job per price list. Most of the time it works. Some nights, one or more of those jobs comes back with a 429 and none of its up to 1000 price records land, silently. BigCommerce allows only one in-flight bulk upsert job per store at a time on the price list records endpoint, no matter which price list each job targets, and a 429 from that lock drops the entire batch rather than applying it partially. Here is why that collision happens and a small script that serializes the writes and retries safely.

`serialize_price_list_upserts.py`

https://www.allanninal.dev/bigcommerce/concurrent-price-list-upserts-429/

https://github.com/allanninal/bigcommerce-fixes/tree/main/concurrent-price-list-upserts-429

---

16 **Concurrent refund requests on one order corrupt payment status**

A support agent double-clicks Refund while an automation script fires the same request. Both calls get accepted. Now the order's payment_status is stuck at Partially Refunded, or over-refunded, and nobody can tell from the admin screen alone what actually happened at the gateway. BigCommerce's refund flow has no per-order lock, so two requests racing on the same order_id can both read the same pre-refund quote and both go through. Here is why that gap exists and a small script that serializes future refunds and flags the orders already caught in the race.

`reconcile_refund_state.py`

https://www.allanninal.dev/bigcommerce/concurrent-refunds-same-order/

https://github.com/allanninal/bigcommerce-fixes/tree/main/concurrent-refunds-same-order

---

17 **BigCommerce coupon applies_to wiped when you edit max_uses**

A merchant needed to bump a coupon's usage cap, so a script sent PUT /v2/coupons/{id} with just {"max_uses": 50}. The response came back 200, and the max_uses field updated fine. What nobody noticed until the coupon started applying storewide was that its product restriction was gone. BigCommerce's legacy Coupons endpoint treats PUT as a full replace, not a patch, so any field you leave out of the body, especially applies_to, gets reset to its default. Here is why that happens and a reconciler that always re-sends the untouched fields so this can not happen again.

`reconcile_coupon_applies_to.py`

https://www.allanninal.dev/bigcommerce/coupon-applies-to-wiped-on-max-uses-edit/

https://github.com/allanninal/bigcommerce-fixes/tree/main/coupon-applies-to-wiped-on-max-uses-edit

---

18 **Coupon usage miscounts in BigCommerce**

A coupon that should still have plenty of uses left suddenly tells a legitimate customer it is maxed out. Nothing looks wrong in the order list. But BigCommerce has been counting every order that ever carried the code, including the ones that were cancelled, declined, refunded, or deleted, and it never gives those uses back. Here is why the count drifts upward and a small script that reconciles it against real completed orders and flags the difference for review.

`reconcile_coupon_usage.py`

https://www.allanninal.dev/bigcommerce/coupon-usage-miscounts/

https://github.com/allanninal/bigcommerce-fixes/tree/main/coupon-usage-miscounts

---

19 **Create customer for an existing email errors without returning the existing id**

A shopper already has a customer record. Something on your side, a sync job, an import, a signup retry, tries to create them again with POST /v3/customers, and BigCommerce correctly refuses it. But the 422 response only says the email is already in use. It never tells you which customer_id already owns that email, so your only path forward is a second lookup call. Here is why the error is shaped that way and a small script that resolves the real id instead of guessing.

`resolve_duplicate_customer.py`

https://www.allanninal.dev/bigcommerce/create-customer-existing-email-no-id-returned/

https://github.com/allanninal/bigcommerce-fixes/tree/main/create-customer-existing-email-no-id-returned

---

20 **BigCommerce customers filter by id is rejected as unsupported**

A script written against BigCommerce's v3 conventions calls GET /v2/customers?id=123 expecting the same id filter that works on v3, and gets back a 400: "The field 'id' is not supported by this resource." The customer record is fine. The v2 Customers list endpoint just never implemented id as a filterable field. Here is why that gap exists and a small script that reconciles the lookup through the direct resource path or migrates it to v3 outright.

`reconcile_customer_id_filter.py`

https://www.allanninal.dev/bigcommerce/customer-filter-by-id-unsupported/

https://github.com/allanninal/bigcommerce-fixes/tree/main/customer-filter-by-id-unsupported

---

21 **BigCommerce customer group change does not immediately refresh cached pricing**

An admin moves a customer into a new pricing group. The customer record updates right away. But the cart the customer already has open, their browser session, or the CDN's cached version of the page keeps quoting the old group's price list for minutes, sometimes longer. BigCommerce resolves customer-group pricing once per cart or session, not on every page view, so nothing tells that stale cart to look again. Here is why that gap opens up and a small script that finds the carts genuinely quoting a stale price and safely forces them to re-resolve.

`refresh_stale_group_pricing.py`

https://www.allanninal.dev/bigcommerce/customer-group-change-stale-price-cache/

https://github.com/allanninal/bigcommerce-fixes/tree/main/customer-group-change-stale-price-cache

---

22 **Customer group mis-assigned**

A wholesale buyer signs up, or a VIP customer crosses a spend threshold, and they are supposed to land in a special customer group with a contracted price. Instead they check out at full retail, and nothing in the admin explains why. Here is why BigCommerce can quietly leave a customer in the wrong customer_group_id, and a small script that computes the group a customer should be in, diffs it against the group they are actually in, and reassigns the wrong ones safely.

`fix_customer_group.py`

https://www.allanninal.dev/bigcommerce/customer-group-mis-assigned/

https://github.com/allanninal/bigcommerce-fixes/tree/main/customer-group-mis-assigned

---

23 **Customer password update intermittently returns random 400 errors**

You call the same endpoint with the same shape of request, and sometimes it works and sometimes it returns a 400 with no pattern you can find. The BigCommerce customers endpoint is a batch array API, it enforces password rules it never shows you, and it caps concurrent requests at 3. A status code alone cannot tell you whether the password actually failed or the write actually landed. Here is why the 400 looks random and a small script that checks the truth before it decides anything.

`recheck_password_updates.py`

https://www.allanninal.dev/bigcommerce/customer-password-update-random-400/

https://github.com/allanninal/bigcommerce-fixes/tree/main/customer-password-update-random-400

---

24 **Declined order still holds stock**

A gateway declined the charge, or a fraud tool pushed the order to Declined, and the order correctly shows no payment. But the inventory it debited when the order was created is often still gone. Nobody paid for it, nobody is shipping it, and it is sitting withheld from your sellable count anyway. Here is why BigCommerce leaves that stock behind and a small script that finds the affected orders and gives the stock back, safely.

`restock_declined_orders.py`

https://www.allanninal.dev/bigcommerce/declined-order-still-holds-stock/

https://github.com/allanninal/bigcommerce-fixes/tree/main/declined-order-still-holds-stock

---

25 **Deleting a product or category leaves a dangling URL with no redirect**

Change a product's custom_url and BigCommerce quietly writes a 301 so the old link keeps working. Delete that same product outright, and nothing gets written at all. The old path just starts 404ing, forever, silently discarding whatever link equity, back-links, and bookmarks were pointing at it. Here is why deletions skip the redirect step entirely and a small script that finds every dangling path and fixes only the ones nothing already covers.

`repair_deleted_url_redirects.py`

https://www.allanninal.dev/bigcommerce/deleted-product-no-redirect/

https://github.com/allanninal/bigcommerce-fixes/tree/main/deleted-product-no-redirect

---

26 **BigCommerce disputed order not flagged**

A customer files a chargeback with their bank. The gateway starts pulling the money back and opens a dispute case. But the order sitting in your BigCommerce admin still shows the same status it always had, Completed, Awaiting Fulfillment, whatever it was before, with nothing anywhere telling you a dispute is in progress. BigCommerce does not watch your gateway for chargebacks on its own, so status_id 13, Disputed, only gets set if a webhook happens to fire and a listener happens to catch it, or a person notices and changes it by hand. Here is why that gap opens up and a small script that reads each order's own transactions and flags the ones that actually need a human's attention.

`flag_disputed_orders.py`

https://www.allanninal.dev/bigcommerce/disputed-order-not-flagged/

https://github.com/allanninal/bigcommerce-fixes/tree/main/disputed-order-not-flagged

---

27 **Customer address create no-ops on an exact duplicate with a 200 response**

A sync job, an import, or a checkout retry posts a new customer address with POST /v3/customers/addresses, and BigCommerce answers with a clean 200. Everything about the response looks like success. Except the address you meant to add was never created, because it already existed, matched field for field, and BigCommerce quietly deduped it instead of erroring or telling you. No id comes back. Nothing in the payload says "no-op." Here is why that gap opens up and a small script that catches it with a before and after snapshot instead of trusting the status code.

`detect_address_noop.py`

https://www.allanninal.dev/bigcommerce/duplicate-address-create-silent-noop/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-address-create-silent-noop

---

28 **Duplicate customers for one email in BigCommerce**

The same shopper checks out as a guest one week and creates an account the next, or gets imported twice from a POS with slightly different spacing in the email, and now their order history is split across two or three customer records. BigCommerce never notices and never merges them on its own. Here is why it happens and a small script that finds every cluster, reassigns the orders to one survivor, and only removes the duplicate once you have confirmed it is safe.

`merge_duplicate_customers.py`

https://www.allanninal.dev/bigcommerce/duplicate-customers-for-one-email/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-customers-for-one-email

---

29 **Duplicate or missing SKUs in BigCommerce**

Two products quietly share the same SKU, or a product has no SKU at all, and nobody notices until an order export breaks or a POS system maps the wrong item. BigCommerce only checks a SKU is unique the moment you write it. It never scans the catalog afterward, so conflicts from CSV imports, sync tools, and product duplication pile up unseen. Here is why it happens and a small script that finds every conflict and flags it for a human to fix.

`find_sku_conflicts.py`

https://www.allanninal.dev/bigcommerce/duplicate-or-missing-skus/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-or-missing-skus

---

30 **Duplicate orders from a double submit**

The gateway took a few extra seconds to respond, or the customer clicked Place Order twice because nothing on screen told them it was working. Either way, BigCommerce now shows two orders with the same items, the same total, and the same customer, created seconds apart. Here is why BigCommerce lets this happen and a small script that finds the pair and cancels the one that should not exist.

`find_duplicate_orders.py`

https://www.allanninal.dev/bigcommerce/duplicate-orders-from-a-double-submit/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-orders-from-a-double-submit

---

31 **Duplicating a product creates variants with colliding SKUs**

You click Duplicate on a product with a dozen variants, expecting a clean copy to edit. Instead every cloned variant is quietly stamped with the same SKU as the original, or no SKU at all, and nothing warns you. BigCommerce only checks SKU uniqueness the moment something writes to it, not the moment a product is cloned, so the collision sits there until an inventory sync, a bulk edit, or a later save trips a 409 Conflict. Here is why the copy skips new SKUs and a small script that finds every collision and safely renames only the duplicates.

`fix_colliding_variant_skus.py`

https://www.allanninal.dev/bigcommerce/duplicate-product-creates-colliding-skus/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-product-creates-colliding-skus

---

32 **Duplicate webhook deliveries run twice**

An order gets fulfilled twice. A confirmation email goes out twice. A downstream system double counts a sale. You check the logs and the same BigCommerce webhook payload really did arrive more than once. Nothing is broken. BigCommerce's webhook dispatcher was designed around retrying until it gets a clean response, not around delivering each event exactly one time, and a couple of ordinary store behaviors can pile more copies on top of that. Here is why it happens and a small, idempotent layer that stops your handler from acting on the same event twice.

`dedupe_webhook_deliveries.py`

https://www.allanninal.dev/bigcommerce/duplicate-webhook-deliveries-run-twice/

https://github.com/allanninal/bigcommerce-fixes/tree/main/duplicate-webhook-deliveries-run-twice

---

33 **Expired promotion still applies**

The sale ended last week, but the discount is still showing up at checkout. A customer gets a price they should not, a report shows a promotion that was supposed to be retired, and nobody changed anything in the admin. BigCommerce's own status field on the promotion is the reason: it does not flip to DISABLED on its own when end_date passes. Here is why that gap exists and a small script that finds every promotion still ENABLED past its end date and disables it safely.

`disable_expired_promotions.py`

https://www.allanninal.dev/bigcommerce/expired-promotion-still-applies/

https://github.com/allanninal/bigcommerce-fixes/tree/main/expired-promotion-still-applies

---

34 **Gateway refund not reflected on the order**

Support refunded the customer straight in Stripe, or the processor pushed the money back on its own. The customer has the funds. But the BigCommerce order still says Completed, or Awaiting Fulfillment, as if nothing happened. Nobody notices until the customer emails asking why the order looks unrefunded, or a report shows revenue that was already given back. Here is why BigCommerce never hears about a refund issued outside its own Refund action, and a small script that finds the orders where that happened and repairs the status safely.

`sync_gateway_refunds.py`

https://www.allanninal.dev/bigcommerce/gateway-refund-not-reflected-on-the-order/

https://github.com/allanninal/bigcommerce-fixes/tree/main/gateway-refund-not-reflected-on-the-order

---

35 **BigCommerce guest orders not linked to accounts**

A shopper checks out as a guest using the same email address they already used to register, or the same email they register with next week. Anyone glancing at the order and the customer record would call these the same person. But BigCommerce never makes that connection itself. The guest order sits at customer_id 0 forever, disconnected from the account, unless someone opens the order in the admin and reassigns it by hand. At scale that means loyalty history, reorder, and lifetime value reports quietly miss every guest purchase whose email happens to match a real account. Here is why that gap never closes on its own and a small script that finds the confident matches and links them safely.

`link_guest_orders.py`

https://www.allanninal.dev/bigcommerce/guest-orders-not-linked-to-accounts/

https://github.com/allanninal/bigcommerce-fixes/tree/main/guest-orders-not-linked-to-accounts

---

36 **Product image upload rejects non fully qualified URLs**

A bulk or scripted import creates the product fine, then the image call comes back with a 422 image_url is invalid error. BigCommerce fetches the image file itself, server-side, from the URL you send it, so a relative path, a root-relative path, or a bare filename gives its fetcher nothing to resolve against. The import job moves on, the product stays real, and it just sits there with zero images. Here is why that happens and a small script that finds every zero-image product and safely resolves the ones it can.

`fix_relative_image_urls.py`

https://www.allanninal.dev/bigcommerce/image-upload-rejects-relative-urls/

https://github.com/allanninal/bigcommerce-fixes/tree/main/image-upload-rejects-relative-urls

---

37 **BigCommerce Inventory API writes are not channel aware**

A product can be assigned to one sales channel, several, or none at all. But BigCommerce stores its inventory_level and inventory_warning_level per product or variant at the catalog level, not per channel, and the bulk Inventory API endpoints have no channel_id parameter to scope a write. So a script that pulls a product list for one channel and then bulk-adjusts stock by product_id, variant_id, or sku can silently change quantities for a product that lives on a completely different channel, or no channel at all. Here is why that gap exists and a pre-flight and post-flight reconciler that catches it before, and after, it happens.

`channel_scoped_inventory.py`

https://www.allanninal.dev/bigcommerce/inventory-api-not-channel-aware/

https://github.com/allanninal/bigcommerce-fixes/tree/main/inventory-api-not-channel-aware

---

38 **BigCommerce variant inventory sum silently fails to save past int32 max**

You PUT a new inventory_level, BigCommerce answers 200, and the number on the variant does not move. Nothing errors. Nothing warns you. The Catalog API stores inventory_level as a 32-bit signed integer with a ceiling of 2147483647, and it checks that ceiling against the product's summed variant inventory, not just the one variant you touched. Once a write would push that sum over the edge, the platform quietly keeps the old value and tells you everything is fine. Here is why that happens and a small check that catches it before you ever trust the response.

`check_inventory_overflow.py`

https://www.allanninal.dev/bigcommerce/inventory-int32-overflow-silent-failure/

https://github.com/allanninal/bigcommerce-fixes/tree/main/inventory-int32-overflow-silent-failure

---

39 **Inventory read immediately after write returns stale stock**

You PUT an inventory adjustment, BigCommerce answers 200 with an action id, and your very next GET still shows the old quantity. Nothing errored. Nothing looks wrong. The Inventory API commits and propagates writes asynchronously, so a read that lands too soon can see the pre-write stock with no signal that it is stale. Here is why that gap opens up and a small script that polls for confirmation with backoff and flags, never silently re-submits, anything that never catches up.

`confirm_inventory_write.py`

https://www.allanninal.dev/bigcommerce/inventory-read-after-write-lag/

https://github.com/allanninal/bigcommerce-fixes/tree/main/inventory-read-after-write-lag

---

40 **Legacy customer group discount blocks a price list from attaching**

You assigned a Price List to a customer group through the V3 API, the assignment record exists, but the storefront still shows the old pricing. Nothing about the assignment call failed. The group just still has a legacy discount configured underneath it, and BigCommerce customer groups only ever run one pricing mechanism at a time. Here is why that collision happens and a small script that finds and clears every group where it is silently blocking a price list.

`clear_blocking_group_discounts.py`

https://www.allanninal.dev/bigcommerce/legacy-group-discount-blocks-price-list/

https://github.com/allanninal/bigcommerce-fixes/tree/main/legacy-group-discount-blocks-price-list

---

41 **BigCommerce line item option valueId type is inconsistent across product option types**

A script reads checkout.cart.lineItems, grabs each option's valueId, and forwards it straight into the order product_options array. For a dropdown it works. For a text field it sends null. For some carts the very same numeric id shows up as a string. The v2 Orders API rejects the mismatched shapes with a single vague error: the options of one or more products are invalid. Here is why valueId has three different shapes depending on the option type, and a normalizer that resolves the real catalog id before it ever reaches the order.

`normalize_line_item_options.py`

https://www.allanninal.dev/bigcommerce/line-item-option-valueid-type-inconsistent/

https://github.com/allanninal/bigcommerce-fixes/tree/main/line-item-option-valueid-type-inconsistent

---

42 **Manual status change to Refunded does not move any money**

An order sits at status_id 4, Refunded, or 14, Partially Refunded. The customer is asking where their money is. You check the transactions endpoint and there is nothing there, no refund entry, no amount, no gateway call. BigCommerce let someone set that status directly, with zero side effects, because order status and money movement are two completely separate systems. Here is why that gap opens up and a reconciler that flags every orphaned Refunded status instead of quietly trusting the label.

`find_orphaned_refund_statuses.py`

https://www.allanninal.dev/bigcommerce/manual-refunded-status-without-transaction/

https://github.com/allanninal/bigcommerce-fixes/tree/main/manual-refunded-status-without-transaction

---

43 **BigCommerce orders stuck on Manual Verification Required, and never cleared**

A fraud-screening app flags an order for review, BigCommerce parks it at status_id 12, and a person then approves it inside that app's own dashboard. Nothing ever tells BigCommerce the review passed, so the order sits at Manual Verification Required forever, blocked from fulfillment, while the human approval it is waiting on already happened somewhere else. Here is why that gap exists and a small script that finds the orders a human already cleared and moves just those, one at a time.

`clear_manual_verification.py`

https://www.allanninal.dev/bigcommerce/manual-verification-required-never-cleared/

https://github.com/allanninal/bigcommerce-fixes/tree/main/manual-verification-required-never-cleared

---

44 **Missed webhooks with no backfill**

Your endpoint was down for a deploy, or it started returning 500s, or a certificate expired. BigCommerce kept trying to deliver, on a backoff schedule, for about 48 hours across up to 11 attempts. Then it gave up for good, flipped the hook's is_active to false, and emailed the address on file. Every store_order_created and store_order_status_updated event from that window is gone. There is no dead letter queue, no event log, and no replay button on BigCommerce's side. Here is why that gap is permanent and a small script that finds it and repairs it by asking the REST API what actually happened.

`backfill_missed_webhooks.py`

https://www.allanninal.dev/bigcommerce/missed-webhooks-with-no-backfill/

https://github.com/allanninal/bigcommerce-fixes/tree/main/missed-webhooks-with-no-backfill

---

45 **Ship to multiple addresses produces inconsistent line item to address mapping**

A shopper splits their cart across three addresses. Two weeks later, support finds a jacket that was supposed to go to their sister's house is missing from every shipping address on the order, while a different item shows up twice. BigCommerce's multi-address checkout builds the order from a sequence of independent API calls against a mutable checkout, and if one of those calls is slow, retried, or skipped, the resulting order can drift from what the customer actually chose, silently and permanently. Here is why that gap opens up and a script that finds the drift so a human can fix it.

`find_consignment_drift.py`

https://www.allanninal.dev/bigcommerce/multi-address-checkout-consignment-drift/

https://github.com/allanninal/bigcommerce-fixes/tree/main/multi-address-checkout-consignment-drift

---

46 **Negative inventory from overselling**

A SKU shows -3 in the admin instead of 0. Nobody typed that number in. It got there because checkout let two orders take the last unit at almost the same moment, or an import wrote a bad delta, or a channel sync double counted a sale. Whatever the cause, BigCommerce keeps selling that SKU exactly like it has real stock, because nothing in checkout stops the count from going below zero. Here is why that happens and a small script that finds every negative SKU and safely resets it back to zero.

`fix_negative_inventory.py`

https://www.allanninal.dev/bigcommerce/negative-inventory-from-overselling/

https://github.com/allanninal/bigcommerce-fixes/tree/main/negative-inventory-from-overselling

---

47 **New BigCommerce storefront channel starts with an incomplete category tree**

You spin up a second storefront channel, expecting the same navigation the primary storefront already has. Instead half the categories are missing, or the tree is entirely empty. Nothing broke. A category tree in BigCommerce's Multi-Storefront setup can only ever belong to one channel, and creating a channel never clones it. Here is why that gap is permanent by default and a small script that finds exactly which category nodes are missing and backfills them safely.

`backfill_category_tree.py`

https://www.allanninal.dev/bigcommerce/new-channel-incomplete-category-tree/

https://github.com/allanninal/bigcommerce-fixes/tree/main/new-channel-incomplete-category-tree

---

48 **No API endpoint to merge two customer records**

The same shopper checks out as a guest, then registers later with a slightly different email casing, and BigCommerce gives you two completely separate customer_id's. Their orders, addresses, and store credit are split across records that the admin UI shows as unrelated people, and there is no merge button and no merge endpoint anywhere in the REST Management API. Here is why that gap exists and a script that safely consolidates the duplicate into one canonical customer.

`merge_duplicate_customers.py`

https://www.allanninal.dev/bigcommerce/no-customer-merge-endpoint/

https://github.com/allanninal/bigcommerce-fixes/tree/main/no-customer-merge-endpoint

---

49 **OAuth token silently stops working after app scopes change**

Everything was fine yesterday. Today every call your script makes to the BigCommerce API returns a plain 401 Unauthorized, nothing else changed in your code, and the token has not been touched. What actually happened is upstream: someone edited the app's declared scopes in the Developer Portal, and BigCommerce quietly invalidated the access token you have been holding onto. There is no distinct error for this. It looks exactly like a revoked or expired token unless you go looking for the difference.

`check_oauth_scope_drift.py`

https://www.allanninal.dev/bigcommerce/oauth-token-invalid-after-scope-change/

https://github.com/allanninal/bigcommerce-fixes/tree/main/oauth-token-invalid-after-scope-change

---

50 **BigCommerce order count endpoint disagrees with actual paginated order list**

One script calls GET /v2/orders/count and gets back a tidy number. Another script pages all the way through GET /v2/orders and counts as it goes. The two totals do not match, and nothing in either response tells you why. The gap is almost never store-data corruption. It is two calls quietly applying two different implicit status_id filters, plus the fact that a count is a snapshot while a multi-page scan takes real time. Here is why that happens and a small reconciler that localizes the mismatch instead of guessing at it.

`reconcile_order_counts.py`

https://www.allanninal.dev/bigcommerce/order-count-endpoint-mismatch/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-count-endpoint-mismatch

---

51 **Order missing shipping address when no line item is flagged physical**

You audit an order, call GET /v2/orders/{id}/shipping_addresses, and get back an empty array. Before you file that as a bug, check what was actually in the cart. BigCommerce only writes a shipping_addresses record when the checkout contained at least one physical line item. An order made up entirely of downloads, services, or gift certificates never gets one, and that is correct. The real anomaly is a physical item with no address on file. Here is how to tell the two apart and a script that flags only the genuine problem.

`flag_missing_shipping_addresses.py`

https://www.allanninal.dev/bigcommerce/order-missing-shipping-address-no-physical-item/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-missing-shipping-address-no-physical-item

---

52 **BigCommerce order-level refund does not recalculate total_tax**

A customer got their money back. The refund shows up in the transaction log. But the order's total_tax never moved, so the order now reports more tax than it actually collected. BigCommerce quietly treats an order-level refund as a flat, tax-exempt custom amount instead of routing it through the tax provider the way a line-item refund does. Here is why that gap opens up and a small script that finds every order where the stored tax has drifted from what the refunds actually say.

`reconcile_refund_tax.py`

https://www.allanninal.dev/bigcommerce/order-refund-does-not-recalc-tax/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-refund-does-not-recalc-tax

---

53 **Order shipments response drops the items array**

The raw shipment JSON from BigCommerce has the shipped lines right there, an items array of order_product_id, product_id, and quantity. But the object your code actually works with shows items as null, empty, or missing entirely. Nothing is wrong on the BigCommerce store. A mapping layer built around the shipment's flat scalar fields quietly left the one nested array out. Here is why that happens and a small reconciler that catches it before it silently breaks your shipped-quantity reporting.

`find_shipment_items_drift.py`

https://www.allanninal.dev/bigcommerce/order-shipments-missing-items-array/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-shipments-missing-items-array

---

54 **Writing order status text instead of status_id fails or no-ops**

The order shows "Awaiting Fulfillment" in the admin, so a script sends {"status": "Shipped"} to move it forward. Nothing happens, or the API rejects the call outright. BigCommerce's V2 Orders resource models order state as a numeric status_id. The status text you see is a read-only label the server computes from that id, not a field you can write to. Here is why that label can never be sent back, and a small script that resolves any status name to its integer status_id before writing.

`write_order_status_id.py`

https://www.allanninal.dev/bigcommerce/order-status-write-requires-status-id/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-status-write-requires-status-id

---

55 **Order tax off by a cent**

The checkout page showed one tax figure. The order that BigCommerce actually persisted shows another, a cent or two apart. Nobody typed in a wrong number. Nobody touched the tax rate. It is BigCommerce's own line-item rounding doing exactly what it is documented to do, just in a way that quietly drifts from what the customer saw on screen. Here is why it happens and a small script that flags the orders where it happened, without touching a single tax total.

`find_tax_mismatch.py`

https://www.allanninal.dev/bigcommerce/order-tax-off-by-a-cent/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-tax-off-by-a-cent

---

56 **BigCommerce order total wrong when only one of price_ex_tax or price_inc_tax is set**

Someone's checkout, ERP sync, or migration script set price_inc_tax on a line item, or total_inc_tax on the order, and left the matching ex_tax field untouched. BigCommerce did not complain. It saved exactly what it was given. Now the order's totals do not add up against tax_total or the line items, and nobody notices until an accounting export or a customer's order history looks wrong. Here is why the V2 Orders API lets this happen and a small script that finds every order it happened to.

`find_tax_override_desync.py`

https://www.allanninal.dev/bigcommerce/order-total-partial-tax-field-override/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-total-partial-tax-field-override

---

57 **Order update call recalculates and overwrites a coupon adjusted total**

A customer checks out with a coupon applied. Weeks later, someone updates the order's staff notes, or a shipping cost, through PUT /v2/orders/{id}. The unrelated field saves fine, but the coupon discount is gone and the total has quietly grown back toward the pre-discount price. BigCommerce has no writable coupon_discount field, so a PUT that touches any total-affecting property recalculates the order from its line items and cost fields and clears the discounts that were riding on top. Here is why that gap opens up and a script that detects and reports it, without guessing at an unsafe auto-fix.

`detect_coupon_overwrite.py`

https://www.allanninal.dev/bigcommerce/order-update-overwrites-coupon-total/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-update-overwrites-coupon-total

---

58 **Order webhooks stop firing entirely with no surfaced error**

Orders keep coming in, the storefront looks fine, and then you notice your fulfillment queue or inventory sync has not heard from BigCommerce in hours. Nothing in the control panel says a webhook died. BigCommerce quietly disables a subscription after it keeps failing to deliver, and separately can blocklist your whole receiving domain for a few minutes at a time if its short-term success rate dips, and neither event shows up as an alert anywhere you would normally look. Here is why that happens and a small script that finds the gap and flags it instead of guessing.

`detect_webhook_gap.py`

https://www.allanninal.dev/bigcommerce/order-webhooks-stop-firing-silently/

https://github.com/allanninal/bigcommerce-fixes/tree/main/order-webhooks-stop-firing-silently

---

59 **Orders stuck Awaiting Shipment past SLA**

The order was paid days ago. It moved to Awaiting Shipment right on schedule. Then nothing. No shipment, no update, no alert, until a customer writes in asking where their package is. BigCommerce never put a clock on that status, so an order can sit there forever if a warehouse task is missed or the system that was supposed to notice has gone quiet. Here is why that silence is allowed to happen and a small script that finds the orders already past your shipping promise.

`find_overdue_awaiting_shipment.py`

https://www.allanninal.dev/bigcommerce/orders-stuck-awaiting-shipment-past-sla/

https://github.com/allanninal/bigcommerce-fixes/tree/main/orders-stuck-awaiting-shipment-past-sla

---

60 **BigCommerce orders stuck on Awaiting Payment after capture**

The gateway shows the money captured. The transaction record on the order agrees. But the order itself still sits at status_id 7, Awaiting Payment, so it never reaches fulfillment. BigCommerce order status and payment status are decoupled from the actual gateway transaction, and the async confirmation that is supposed to move the order along can go missing. Here is why that gap opens up and a small script that finds the orders truly captured and safely advances only those.

`advance_captured_orders.py`

https://www.allanninal.dev/bigcommerce/orders-stuck-on-awaiting-payment-after-capture/

https://github.com/allanninal/bigcommerce-fixes/tree/main/orders-stuck-on-awaiting-payment-after-capture

---

61 **Webhook keeps firing after its owning app or token is removed**

The app is gone. The token was revoked weeks ago. But the destination endpoint keeps getting POSTed to on every order and cart event, and nobody can figure out from the BigCommerce admin why. Uninstalling an app through the Marketplace, or deleting an API account in the control panel, is supposed to cascade-delete its webhooks. Anything outside that clean path leaves them behind, still firing, still eating into the ten-webhook-per-client_id quota. Here is why the orphan slips through and a small reconciler that finds and safely clears it.

`reconcile_orphaned_webhooks.py`

https://www.allanninal.dev/bigcommerce/orphaned-webhooks-after-token-removal/

https://github.com/allanninal/bigcommerce-fixes/tree/main/orphaned-webhooks-after-token-removal

---

62 **Out of stock but still purchasable**

The admin shows zero units left. The warehouse agrees, there is nothing on the shelf. But the storefront still has an Add to Cart button, and the API still accepts the order. Here is why BigCommerce sometimes never looks at a SKU's stock at all, and a small script that finds every product where checkout should be blocked but is not, and flags it for a human instead of guessing.

`find_stale_in_stock.py`

https://www.allanninal.dev/bigcommerce/out-of-stock-but-still-purchasable/

https://github.com/allanninal/bigcommerce-fixes/tree/main/out-of-stock-but-still-purchasable

---

63 **Manually overridden order pricing is excluded from promotions**

An integration creates an order with an explicit price on a line item, a server-to-server checkout, a custom quote, a marketplace import, and the order total comes back with zero promo discount even though an active, matching promotion is running. Nothing errors. The coupon field is just quietly empty. BigCommerce's pricing engine only evaluates promotions against prices its own pricing service computed, and a manually overridden price does not qualify by default. Here is why that gap exists and a small script that finds every order it happened to.

`flag_overridden_pricing_promotions.py`

https://www.allanninal.dev/bigcommerce/overridden-order-pricing-excludes-promotions/

https://github.com/allanninal/bigcommerce-fixes/tree/main/overridden-order-pricing-excludes-promotions

---

64 **Paid order stuck on Incomplete**

The gateway shows a successful capture. The money is real. But the BigCommerce order still sits at status_id 0, Incomplete, invisible to your fulfillment queue and to most order exports and webhooks. Here is why that gap opens up and a small script that finds the confirmed paid orders and moves them forward safely.

`reconcile_incomplete_orders.py`

https://www.allanninal.dev/bigcommerce/paid-order-stuck-on-incomplete/

https://github.com/allanninal/bigcommerce-fixes/tree/main/paid-order-stuck-on-incomplete

---

65 **A parent promotion's max_uses overrides a coupon's own usage cap**

A coupon code still looks available. Its own max_uses has plenty of headroom left. But the shopper types it in at checkout and gets invalid coupon code anyway. Nothing about the coupon record itself changed. The parent Promotion it lives under has its own, separate max_uses cap, and BigCommerce checks that cap first. Here is why the promotion-level limit silently overrides a coupon that looks fine in isolation, and a small script that flags every code this is happening to.

`find_capped_promotion_codes.py`

https://www.allanninal.dev/bigcommerce/parent-promotion-caps-override-coupon/

https://github.com/allanninal/bigcommerce-fixes/tree/main/parent-promotion-caps-override-coupon

---

66 **Partial shipment total mismatch**

A WMS or 3PL posted shipments in batches as pickers worked through the warehouse, and now the order does not add up. It says Partially Shipped when every unit clearly left the building, or a line shows a shipped quantity that does not match what was ordered or what got refunded. The customer already has their package, but BigCommerce's own bookkeeping disagrees with itself. Here is why the numbers drift apart and a small script that finds the mismatched orders, safely fixes the ones that are safe to fix, and flags the rest for a human.

`find_shipment_mismatch.py`

https://www.allanninal.dev/bigcommerce/partial-shipment-total-mismatch/

https://github.com/allanninal/bigcommerce-fixes/tree/main/partial-shipment-total-mismatch

---

67 **Presentment vs settlement currency**

A shopper in Berlin checks out in EUR. The store's books run in USD. The bank deposit that shows up two days later is a third number, in a currency and at a rate the payment gateway chose on its own schedule. All three of these, the presentment amount the shopper saw, the order total BigCommerce stored, and the settlement amount that landed in the bank, can legitimately disagree. Naive reconciliation that just matches order total to bank deposit breaks quietly and misreports revenue or FX gain and loss. Here is why it happens and a small script that flags the variance without touching a single money field on the order.

`find_currency_variance.py`

https://www.allanninal.dev/bigcommerce/presentment-vs-settlement-currency/

https://github.com/allanninal/bigcommerce-fixes/tree/main/presentment-vs-settlement-currency

---

68 **Price list changes fire no product or SKU webhooks**

A price list record's price changes. The storefront starts showing the new number. But no store/product/updated event lands, no store/sku/updated event lands, and a catalog-sync integration that only listens for those two scopes never learns the price moved at all. BigCommerce Price Lists are a pricing overlay resolved at cart and storefront time, not a write to the product or variant row, so the change fires its own separate webhook family instead, one most integrations never subscribed to. Here is why that gap opens up and a script that finds every price list change your webhooks missed.

`detect_price_list_webhook_gap.py`

https://www.allanninal.dev/bigcommerce/price-list-changes-fire-no-webhooks/

https://github.com/allanninal/bigcommerce-fixes/tree/main/price-list-changes-fire-no-webhooks

---

69 **Price list has no entry for a variant so it falls back to catalog price**

Every other variant of the product shows the discounted price. One SKU quietly charges full catalog price instead, and nothing in the admin tells you why. BigCommerce price lists store overrides as flat per-variant records, not product-level rules that cascade to children, so a CSV import or API upsert that misses one variant leaves a silent gap. Here is why that gap opens up and a small script that finds every variant a price list forgot.

`price_list_variant_gaps.py`

https://www.allanninal.dev/bigcommerce/price-list-missing-variant-entry/

https://github.com/allanninal/bigcommerce-fixes/tree/main/price-list-missing-variant-entry

---

70 **Price list not applied to a group**

A wholesale group is supposed to see special prices. The price list exists, the records are there, everything looks configured. But the group's customers still check out at the regular catalog price, and nothing in the admin says why. Here is why a BigCommerce price list can sit right next to a customer group and never actually apply to it, and a small script that finds the missing link and reconnects it safely.

`fix_price_list_assignment.py`

https://www.allanninal.dev/bigcommerce/price-list-not-applied-to-a-group/

https://github.com/allanninal/bigcommerce-fixes/tree/main/price-list-not-applied-to-a-group

---

71 **Product invisible on a channel despite correct category and visibility flags**

The product is visible, correctly categorized, and sells fine on your original storefront. On a second or third channel it 404s or is simply absent from listings and search. Category membership and is_visible were never the gatekeeper for a sales channel. A separate table decides whether a channel can see the product at all, and nothing fills it in automatically when you spin up a new storefront. Here is why that gap opens up and a script that finds every product missing from every channel's assignment set.

`find_missing_channel_assignments.py`

https://www.allanninal.dev/bigcommerce/product-invisible-missing-channel-assignment/

https://github.com/allanninal/bigcommerce-fixes/tree/main/product-invisible-missing-channel-assignment

---

72 **Products stranded with no category**

A product was created through the API, a bulk import, or a migration script, and it saved without an error. It has a name, a price, a SKU, everything looks right in the admin. But no shopper ever finds it by browsing. It never shows up under any category, any navigation link, any facet. The product is not broken. It is just invisible, because nothing ever pointed a category at it. Here is why BigCommerce quietly lets this happen and a small script that finds every stranded product and gives it a fallback category so it can be found again.

`fix_stranded_products.py`

https://www.allanninal.dev/bigcommerce/products-stranded-with-no-category/

https://github.com/allanninal/bigcommerce-fixes/tree/main/products-stranded-with-no-category

---

73 **BigCommerce promotion with both group_ids and excluded_group_ids never triggers**

The promotion saved fine. The API answered 200. But at checkout, nobody gets the discount, not even the VIP shoppers the merchant meant to include. BigCommerce's Promotions API lets a promotion's customer eligibility object carry both an allow-list of groups and a deny-list of groups at the same time, and when both are populated, the promotion engine has no defined rule for which one wins, so it fails closed for everyone. Here is why that combination is dead on arrival and a small script that finds every promotion carrying it.

`find_group_conflicts.py`

https://www.allanninal.dev/bigcommerce/promotion-group-exclusion-conflict/

https://github.com/allanninal/bigcommerce-fixes/tree/main/promotion-group-exclusion-conflict

---

74 **BigCommerce rate limit callback fires only once per session instead of per request**

A script wires a callback into its BigCommerce client to warn it when the rate limit is close, and the callback does fire, exactly once, then never again. Meanwhile the requests keep going and the 429 Too Many Requests responses pile up. BigCommerce never sends a webhook for this. It reports live quota state on every single response through four headers, and the fix is to stop trusting a one-shot callback and start reading those headers on every request.

`rate_limit_guard.py`

https://www.allanninal.dev/bigcommerce/rate-limit-callback-fires-once/

https://github.com/allanninal/bigcommerce-fixes/tree/main/rate-limit-callback-fires-once

---

75 **Refunded line items are not returned to inventory automatically**

A customer gets refunded, the order shows the money back, and the product page still says the same quantity is available as before the sale, or worse, still zero even though the item never left the shelf. BigCommerce refunds are a payment operation only. Nothing about processing a refund tells the catalog that a unit came back. Here is why that gap opens up and a small reconciler that restocks only the refunds that are genuinely safe to restock.

`restock_refunded_inventory.py`

https://www.allanninal.dev/bigcommerce/refund-does-not-restock-inventory/

https://github.com/allanninal/bigcommerce-fixes/tree/main/refund-does-not-restock-inventory

---

76 **Refund rejected on orders paid with split payment methods**

A customer paid part gift card, part credit card, or store credit plus PayPal. Now a refund on that order comes back with "the requested refund had invalid split payment," and the money never moves. BigCommerce settled that order as separate transactions against separate providers, each capped at what it actually captured, and the refund endpoint will not guess how to split a lump sum across them. Here is why the rejection happens and a small script that always asks for the exact split first.

`refund_split_payment.py`

https://www.allanninal.dev/bigcommerce/refund-invalid-split-payment/

https://github.com/allanninal/bigcommerce-fixes/tree/main/refund-invalid-split-payment

---

77 **BigCommerce shipment tracking never added**

A customer emails asking where their package is. You open the order and it clearly says Shipped, but the tracking link in the confirmation email goes nowhere, because there is no tracking number on file. This is not a bug. BigCommerce's shipment resource never required a tracking number in the first place, so the Ship Items modal, and any OMS or 3PL app wired up to it, can create a shipment or flip the order to Shipped without ever asking for one. Here is why that gap is allowed to happen and a small script that finds the orders it already happened to.

`find_untracked_shipped_orders.py`

https://www.allanninal.dev/bigcommerce/shipment-tracking-never-added/

https://github.com/allanninal/bigcommerce-fixes/tree/main/shipment-tracking-never-added

---

78 **Order shipping address update does not recompute tax or shipping cost**

You call the API to fix a typo'd shipping address on an existing order. The address record updates cleanly. But total_tax and shipping_cost never move, because BigCommerce never re-runs the shipping-rate lookup or the tax engine on an order object, only inside cart and checkout consignment flows. The order now points at one destination and bills for another. Here is why that gap exists and a small script that finds the orders it happened to and repairs them safely.

`recompute_stale_totals.py`

https://www.allanninal.dev/bigcommerce/shipping-address-update-stale-totals/

https://github.com/allanninal/bigcommerce-fixes/tree/main/shipping-address-update-stale-totals

---

79 **BigCommerce SKUs endpoint truncates at 50 records without paginating**

A product has 80 variants. A script calls the SKUs endpoint once, gets a tidy array back, and moves on. That array has exactly 50 rows in it. Nothing in the call itself says anything is missing. BigCommerce's catalog SKU and variant sub-resource endpoints are paginated collections where the limit query parameter silently defaults to 50 per page when you leave it out, capped at 250, and the API never auto-paginates or warns you. Here is why that gap opens up and a small reconciler that finds every product where it happened and re-fetches the complete list.

`reconcile_truncated_skus.py`

https://www.allanninal.dev/bigcommerce/sku-endpoint-truncates-at-50/

https://github.com/allanninal/bigcommerce-fixes/tree/main/sku-endpoint-truncates-at-50

---

80 **Stale customer group cached in checkout state after a mid session change**

An admin moves a shopper into a new customer group, or a B2B company role change fires, in the middle of their session. Checkout does not notice. BigCommerce's checkout-sdk-js caches the customer's group id once when checkout state initializes, and its state-merge logic does not reliably overwrite that cached value when a fresher one arrives. Since group pricing is resolved through Price Lists tied to a customer_group_id, the shopper can complete checkout priced under their old, stale group instead of the one they actually belong to now. Here is why the cache goes stale and a script that finds the orders it actually happened to.

`flag_stale_group_orders.py`

https://www.allanninal.dev/bigcommerce/stale-customer-group-in-checkout-state/

https://github.com/allanninal/bigcommerce-fixes/tree/main/stale-customer-group-in-checkout-state

---

81 **Stale product modifiers after import in BigCommerce**

A migration tool rewrites your variants, and the modifier picker in the storefront still lists an option that points at nothing. BigCommerce's CSV import and export can only edit the price or weight adjuster on a modifier option_value that already exists. It cannot create, delete, or re-link one. So when a bulk import deletes and recreates variants, or swaps the product behind a product_list modifier, the old modifier survives on the parent product as an orphan that still renders in the admin and at checkout. Here is why it happens and a small script that finds every stale modifier and reports it for a human to fix.

`find_stale_modifiers.py`

https://www.allanninal.dev/bigcommerce/stale-product-modifiers-after-import/

https://github.com/allanninal/bigcommerce-fixes/tree/main/stale-product-modifiers-after-import

---

82 **Status change skips side effect actions like capture or void**

An order gets marked Refunded, or Cancelled, or Completed, and the status looks right in the admin. But nothing was actually refunded, voided, or captured in the payment gateway. BigCommerce's status_id field is only a label on the order record. It has no hook back into the gateway. Only the Action menu, Refund, Void transaction, Capture funds, actually talks to the payment processor and updates status_id as a side effect. Write status_id directly and you get the label with none of the money movement. Here is why that gap opens up and a script that flags every order where the status claims an action that never happened.

`find_status_without_payment_action.py`

https://www.allanninal.dev/bigcommerce/status-change-skips-payment-side-effects/

https://github.com/allanninal/bigcommerce-fixes/tree/main/status-change-skips-payment-side-effects

---

83 **Status webhook payload carries only an id, hiding dropped updates**

A store/order/statusUpdated webhook fires. Your endpoint receives it, returns 200 OK, and BigCommerce considers the job done. But the payload never told you what the new status actually is, only which order changed. If the follow-up call you make to find out fails, times out, or your app crashes before it finishes, the notification is gone. There is no record that anything changed, no queue to replay from, and BigCommerce will never resend it because as far as it is concerned, delivery already succeeded. Here is why that gap exists and a small reconciler that finds the orders it left behind.

`reconcile_order_status.py`

https://www.allanninal.dev/bigcommerce/status-webhook-payload-missing-detail/

https://github.com/allanninal/bigcommerce-fixes/tree/main/status-webhook-payload-missing-detail

---

84 **Storefront GraphQL variant inventory returns flaky values**

One query says a variant has 12 units available to sell. The next query, seconds later, says 4. The Management API, the actual source of truth, says the real number is 4 the whole time. The Storefront GraphQL API's inventory fields sit behind caching layers and a default-location aggregation rule, and either one can make availableToSell disagree with reality without anything being broken in the data itself. Here is why the number flickers and a small reconciler that flags the real mismatches instead of guessing at a fix.

`reconcile_variant_inventory.py`

https://www.allanninal.dev/bigcommerce/storefront-graphql-flaky-inventory/

https://github.com/allanninal/bigcommerce-fixes/tree/main/storefront-graphql-flaky-inventory

---

85 **Test orders counted in sales reports**

Someone on the team placed a checkout to test a new tax rule, a shipping zone, or a promotion, using the Test Payment Gateway or a real gateway left in sandbox mode. The order that resulted looks completely normal to BigCommerce. It has a revenue-counted status_id, it shows up in Store Overview, and it inflates the Sales report until someone notices the numbers do not match the bank. Here is why BigCommerce cannot tell a test checkout from a real one on its own, and a small script that finds the likely test orders and flags them without touching revenue.

`flag_test_orders.py`

https://www.allanninal.dev/bigcommerce/test-orders-counted-in-sales-reports/

https://github.com/allanninal/bigcommerce-fixes/tree/main/test-orders-counted-in-sales-reports

---

86 **Transactions total does not match the order**

The order says one thing. The gateway's transaction ledger says another. A refund went through in the payment processor's own dashboard, a partial refund never got posted back as a transaction, or a refund_quote amount got overridden on the way out. Now total_inc_tax and refunded_amount on the BigCommerce order do not tie out against what the transactions feed actually shows, and nobody notices until a customer or an accountant asks why the numbers do not add up. Here is why it happens and a small script that flags the orders where it happened, without touching a single total.

`find_transactions_mismatch.py`

https://www.allanninal.dev/bigcommerce/transactions-total-does-not-match-the-order/

https://github.com/allanninal/bigcommerce-fixes/tree/main/transactions-total-does-not-match-the-order

---

87 **BigCommerce uninstall webhook registration silently rejected**

The app calls the BigCommerce hooks API to subscribe to the uninstall event, gets back a 400, and moves on without anyone noticing. Months later a merchant uninstalls the app and nothing happens: no cleanup, no notification, no record that they ever left. The cause is almost always a single wrong character in the scope string. Here is why BigCommerce rejects it outright instead of registering something broken, and a small script that finds every store missing a working uninstall hook and safely re-registers it.

`repair_uninstall_webhook.py`

https://www.allanninal.dev/bigcommerce/uninstall-webhook-registration-rejected/

https://github.com/allanninal/bigcommerce-fixes/tree/main/uninstall-webhook-registration-rejected

---

88 **V3 pagination breaks when options or modifiers are included**

You send limit=250 and add include=options,modifiers so you get variants and modifiers hydrated in the same call. BigCommerce quietly ignores your limit and hands back 10 records a page instead of 250, but meta.pagination.total_pages is still calculated as if your 250 had been honored. Any script that stops walking pages once it hits total_pages stops early, and a chunk of your catalog never makes it into the sync. Here is why the metadata lies and a small script that detects it and works around it.

`check_include_pagination.py`

https://www.allanninal.dev/bigcommerce/v3-pagination-breaks-with-includes/

https://github.com/allanninal/bigcommerce-fixes/tree/main/v3-pagination-breaks-with-includes

---

89 **Variant inventory not tracked**

A product has real size or color options built through the control panel or a bulk import. Each variant even shows a stock number in the admin. But orders keep going through for a SKU that should have been out of stock weeks ago. The count never moves. Here is why BigCommerce quietly ignores a variant's inventory and a small script that finds the affected products and flips the one setting that fixes it, only when it is safe to do so.

`fix_variant_inventory_tracking.py`

https://www.allanninal.dev/bigcommerce/variant-inventory-not-tracked/

https://github.com/allanninal/bigcommerce-fixes/tree/main/variant-inventory-not-tracked

---

90 **Variant price override stops following base product price changes**

A merchant raises the product's base price, and every plain variant follows along on the storefront, except the ones that already had a number typed into their own price field. Those stay frozen at whatever they were set to, silently, with no warning from the API. This is not a bug. It is how BigCommerce's price inheritance is designed to work, and it quietly strands a variant's price the moment anyone, human or script, sets it explicitly. Here is why that split happens and a small script that finds every variant stuck like this so a merchant can decide what to do about it.

`find_stale_variant_prices.py`

https://www.allanninal.dev/bigcommerce/variant-price-override-breaks-inheritance/

https://github.com/allanninal/bigcommerce-fixes/tree/main/variant-price-override-breaks-inheritance

---

91 **Webhook deactivated after failures**

Order and cart events used to show up in your app right away. Now an ERP sync is stale, or a fulfillment queue is empty, and nobody can say why. Nothing errored in the store admin. The webhook just quietly stopped. BigCommerce gave up retrying it a while back, flipped it off, and emailed an address nobody was watching. Here is why that happens and a small script that finds the deactivated or missing hooks and repairs them without flooding a still-broken destination.

`reconcile_webhooks.py`

https://www.allanninal.dev/bigcommerce/webhook-deactivated-after-failures/

https://github.com/allanninal/bigcommerce-fixes/tree/main/webhook-deactivated-after-failures

---

92 **Webhook domain blocklisted after low delivery success ratio**

Deliveries to a domain that was working fine a minute ago suddenly stop, for every hook registered on that host, not just the one that was failing. BigCommerce tracks a rolling success ratio per destination domain, and once it dips below 90 percent it blocklists the whole domain for 3 minutes. There is no API call to lift that block early. Here is why it happens, how to detect it from your own request log, and the one safe repair that is left once the dust settles.

`webhook_domain_health.py`

https://www.allanninal.dev/bigcommerce/webhook-domain-blocklisted-low-success-ratio/

https://github.com/allanninal/bigcommerce-fixes/tree/main/webhook-domain-blocklisted-low-success-ratio

---

93 **BigCommerce webhook fires duplicate events within the same second**

Your endpoint logs two store/order/statusUpdated calls for the same order, same status, a fraction of a second apart. Nothing in your store looks broken, the order's real status is correct, but your handler ran twice. BigCommerce's webhook service is at-least-once, not exactly-once, and a single admin action can legitimately trigger more than one subscription at once. Here is why that happens and a small idempotency check that drops the duplicate without ever touching order state.

`dedupe_same_second_webhooks.py`

https://www.allanninal.dev/bigcommerce/webhook-duplicate-events-same-second/

https://github.com/allanninal/bigcommerce-fixes/tree/main/webhook-duplicate-events-same-second

---

94 **Webhook payload not verified**

Your app registered a webhook, BigCommerce POSTs order and product events to the destination URL, and the handler trusts whatever body shows up. It looks fine until someone finds that URL and sends a payload of their own. BigCommerce webhook callbacks are not signed with a documented, verifiable HMAC the way some other platforms sign their webhooks, so a handler that trusts the JSON body without checking anything else has no real way to know the request came from BigCommerce at all. Here is why that gap exists and a small check that closes it with the one safeguard BigCommerce actually supports.

`verify_webhook_secret.py`

https://www.allanninal.dev/bigcommerce/webhook-payload-not-verified/

https://github.com/allanninal/bigcommerce-fixes/tree/main/webhook-payload-not-verified

---